

# Onyx — Project Overview

*Onyx · multi-process OS on Raspberry Pi 4*

## 1. In one sentence

Onyx is a **small multi-process operating system** for the **Raspberry Pi 4** (AArch64), built **on top of Circle** ([rsta2/circle](#)), which serves as its HAL and driver stack. It loads **ELF programs from the SD card** and runs them as **applications isolated from one another**, each in its own page table. The whole thing is driven by the **Onyx desktop** — a compositor, a menu bar, a dock, a terminal, a file manager — and some ninety applications and services ([sdcard/apps](#)), from an office suite and a web browser to emulators. Our own code is under the MIT licence ([LICENSING.md](#)).

It **runs on real Raspberry Pi 4 hardware** (not just in emulation).



*The Onyx*

*desktop, a modernised CDE: the menu bar, the agenda on the Voronoi wallpaper, a calculator behind (grey frame), the terminal in front (its frame in the theme's colour), the dock with its Internet drawer open. (The real apps, run on a PC by the desktop simulator.)*

## 2. Philosophy: "own the top, reuse the bottom"

Circle is a *single-process, single-address-space, cooperative* bare-metal framework ("your application **is** the system"). Onyx **keeps the bottom half** of Circle (which is excellent) and **replaces its "operating system" half**:

| Layer                                                                         | Decision                                        | Detail                                                                        |
|-------------------------------------------------------------------------------|-------------------------------------------------|-------------------------------------------------------------------------------|
| Reset → EL1, MMU activation, GIC, C++ constructors                            | <b>Keep Circle</b>                              | Our entry point is <code>main()</code> ; everything below it is already done. |
| 64 KB page allocator ( <code>palloc/pfree</code> )                            | <b>Keep</b> — it's our frame allocator          | <code>CMemorySystem::Get()</code> .                                           |
| GIC-400, timer, mailbox, EMMC+FatFs, USB, Ethernet, framebuffer               | <b>Reuse</b>                                    | This is the whole point of building on Circle.                                |
| <code>CScheduler/CTask</code> scheduler                                       | <b>Rewrite the implementation, keep the API</b> | See <a href="#">kernel internals</a> .                                        |
| Per-process page tables, context switching, exception vectors, address spaces | <b>Write from scratch</b>                       | Doesn't exist in Circle.                                                      |

Practical consequence: we **link** the kernel against Circle's already-compiled static libraries (`libcircle.a`, `libfs`, `libusb`, ...) rather than copying its sources.

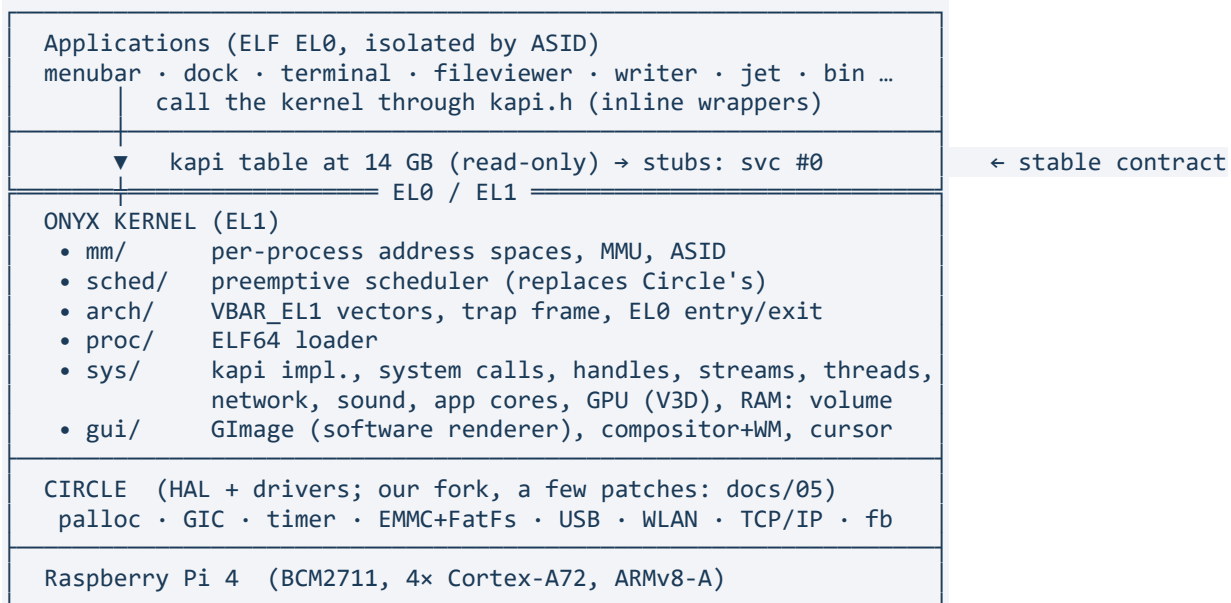
### 3. Main features

- **Per-process isolation via the MMU.** Each application has its own L2/L3 page table (64 KB granule), tagged by **ASID**. One application cannot see another's memory.
- **Protected apps (ELO).** Applications run at **ELO**, the unprivileged level, and call the kernel by **system calls** (`svc`): they are isolated from one another **and** from the kernel and the hardware. An app that crashes is killed; the system goes on. See §5.
- **Stable fixed-address ABI (`kapi`).** Each application sees a **table of function pointers** at a fixed virtual address (14 GB), mapped read-only; each entry is a small stub that makes the system call. Applications call the kernel through this table → **an application binary keeps working without recompilation** when the kernel changes (*append-only* contract, current ABI version: **74**).
- **Threads, app cores, RAM volume.** An app may run threads (mutexes, events, a `futex`, "real time" priority), take a whole CPU core for its own code (the emulators, Doom), and keep files in memory on `RAM:`.
- **The GPU.** The VideoCore VI's 3D unit (V3D) is driven by the kernel for the apps: 3D (the N64 and GameCube emulators, the BASIC's 3D), and the compositing of layers (`user/gpucomp`: Paint's blend modes, Jet Browser).
- **Full graphical desktop.** 32-bit software compositor, window manager, toolkit of kernel-drawn widgets (buttons, checkboxes, sliders, text fields, scroll bars, icons...), windows with themeable decoration, wallpaper, mouse cursor.
- **Onyx shell.** A menu bar (the active app's menus, the clock, the Wi-Fi and volume menus), a dock (drawers of apps, workspaces), notifications, a shared clipboard with a history, an interactive terminal with **pipes and redirection** (`|`, `>`, `>>`, `<`), a column file browser, a Control Panel, a first-run wizard (Setup), and some sixty command-line tools in `/bin`.
- **Application catalog** ([User guide §12](#)). Office: Writer (word processor, PDF export), Spreadsheet, Cardfile (a small database), Ledger (accounting), Calendar, PDF Viewer (MuPDF), RTF reader,

Archiver. Internet: Jet Browser, Mail, IRC, Courier (HTTP client), Lisa (an AI chat). Media: Paint, Photos, the Media Player (music and video), Screenshot, Koton (a music studio with plugins), FM Tracker. Emulators: Game Boy / Color, GBA, NES, SNES, N64, GameCube, with a Game Library. Games (Doom on Freedoom, Tetris, Solitaire, FreeCell, Invaders, Arkanoid...), a BASIC with its editor (QBasic), the Package Manager, a Task Manager, demos.

- **USB input devices.** Keyboard and mouse (HID), USB gamepads and MIDI keyboards, with keyboard layouts loaded from the card (US, UK, DE, FR, BE, ES, IT, Dvorak).
- **Networking (WLAN).** TCP/IP stack + on-board Wi-Fi (BCM4343 / `wpa_supplicant`), on core 3 (`netcore=1`, the shipped card) or core 0; TCP sockets exposed to apps through the ABI, TLS in user space, NTP clock synchronisation, and remote access: a telnet shell, VNC, a window-level remote desktop for Windows (`rdp` + Onyx Remote), an FTP server.
- **Packages.** Every app is a signed package of the `onyx-packages` repository: `pkg`, the Package Manager and an update daemon keep a card up to date.
- **A modern web browser. Jet Browser** (`jet`), the Onyx web browser based on NetSurf: `http://` and `https://`, **CSS3** (`calc()`, `var()`, flexbox, grid, gradients, shadows, rounded corners, gradient text, vendor prefixes), web fonts (WOFF/WOFF2, variable fonts) with Chrome's Windows fonts stood in by metric-compatible ones, and **JavaScript** on QuickJS (ES2023, the DOM, the page laid out again after a script's changes). Details in [Jet Browser, the NetSurf changes](#). Video and audio too (`<video>`, `<audio>`, MSE).

## 4. The architecture at a glance



## 5. The execution model: apps at EL0

The most structurally defining architectural decision. Since kapi v74 (2026-10-02):

- Applications run at **EL0**, **each in its own page table** tagged by ASID; the kernel's memory, the devices and the other apps' pages are out of their reach (EL1-only mappings).
- They **call the kernel through the kapi table** at a fixed address, as before — but in an app that table points at small stubs (`mov x8, #slot; svc #0; ret`): every kapi is a **system call**, its pointers

checked, its handles per process. `memcpy` / `memset` and the event pump run in the app itself (no system call). Existing binaries kept working unchanged.

- **A fault in an app kills that app** (a notice on the desktop, a line in `kmsg`), never the machine.
- History: the project first ran apps at EL1 with direct calls ("Option C", `ARCHITECTURE.md` §11–§12: apps isolated from one another but not from the kernel); the move to EL0 was done in steps (`ELO-PROTECTED-MODE.md`) and the EL1 mode removed.

## 6. Scheduling: preemptive applications, non-preemptive kernel

A **100 Hz timer tick** gives each task a time slice (20 ms); when it expires, the kernel **preempts the application** and runs the next task. An application stuck in a loop that never yields no longer freezes the system: the cursor, the desktop and the other apps stay responsive, and `taskman` can stop it (the `spin` app tests this).

An IRQ taken while an app runs (at EL0) arrives on that task's own kernel stack, with the app's full register and FP state saved there; when the slice is over the kernel yields right there, like a voluntary switch.

Only **application code** is preempted: an app inside a `kapi_*` call, and the kernel's own threads, are not (the **kernel is non-preemptive**, the classic Unix model); its long operations (file reads and writes, SD-card waits) yield between pieces. Tasks still also switch **voluntarily** (`yield`, `msleep`, `present`, `wait`, ...), and a dynamic priority keeps CPU hogs from starving the tasks that yield (the network, the sound). Details in [Kernel internals](#) (§ "Preemptive scheduling").

An application may run **threads** (`kapi v67`): more tasks in its own address space, with mutexes, events and barriers, and calls posted to its event pump — a thread waits on the network while the window stays responsive. The process ends with its main thread; killing it kills them all. The scheduler's task list has no fixed limit.

## 7. Target hardware

- **Raspberry Pi 4** (BCM2711, 4× Cortex-A72, ARMv8-A) — primary target.
- **HDMI** output (32 bpp framebuffer, 1024×768 by default, configurable).
- **USB keyboard + mouse**.
- Optional **serial console** (GPIO14/15, 115200 8N1) for the boot log and exception dumps.
- **The four cores, used asymmetrically by design** (not a symmetric SMP kernel): **core 0** runs the kernel, the scheduler, the desktop and every process; **core 1** the sound producer (and, on a hang, the crash record); **cores 2 and 3** are **app cores**, each lent whole to one app for its own code (an emulator's machine, Doom's engine); with `netcore=1` **core 3** runs the network stack instead. Nothing of the kernel, Circle's drivers or FatFs has to be multi-core safe. (`docs/02` §12, §14, §11.)
- The RPi 5 is not supported yet (I/O behind the RP1 chip via PCIe — heavier bring-up; the plan: `PI5-PORT.md`).

## 8. Repository structure

|                              |                                                                                                                                                                                                                                                   |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>README.md</code>       | what Onyx is, how to build / stage / test, where the docs are                                                                                                                                                                                     |
| <code>ARCHITECTURE.md</code> | the original design + build manifest (historical record)                                                                                                                                                                                          |
| <code>docs/</code>           | THIS documentation (overview, internals, dev/user guides, plans)                                                                                                                                                                                  |
| <code>kernel/</code>         | the Onyx kernel (see <code>docs/02-KERNEL-INTERNALS.md</code> )                                                                                                                                                                                   |
| <code>user/</code>           | the userland: apps ( <code>Apps/&lt;name&gt;/</code> ), wtk toolkit, libraries, <code>/bin</code> tools ( <code>bin/</code> ), runtime ( <code>crt0.S</code> , <code>user.ld</code> , <code>libc/</code> ), Jet Browser ( <code>netsurf/</code> ) |
| <code>sdcard/</code>         | ready-to-flash files for an RPi 4 (firmware, config, apps, <code>/bin</code> , samples)                                                                                                                                                           |
| <code>sdcard_lite/</code>    | the minimal card (the required packages), the rest from the repository                                                                                                                                                                            |

|              |                                                                                                      |
|--------------|------------------------------------------------------------------------------------------------------|
| third_party/ | vendored libraries (NetSurf, FreeType, MuPDF, FFmpeg, QuickJS, ...)                                  |
| tools/       | host scripts and tests (packages, screenshots, desktop simulator, ...)                               |
| pc/          | the Windows builds (Koton, the emulators, Onyx Remote)                                               |
| circle/      | Circle, as a git submodule (the fork <a href="#">stephaneweg/circle</a> , branch <code>onyx</code> ) |

**Legacy docs.** `ARCHITECTURE.md` is the historical design record (its banner says what changed since); where it contradicts this documentation, **docs/ is authoritative**. Its §11–§12 remain the reference for *why* the first execution model ("Option C", apps at EL1) was chosen; apps run at EL0 since kapi v74 (§5).

## 9. Further reading

| Document                            | For whom                                  | Contents                                                               |
|-------------------------------------|-------------------------------------------|------------------------------------------------------------------------|
| <b>02 — Kernel Internals</b>        | anyone who wants to understand the kernel | boot, memory/MMU, scheduling, exceptions, ABI, GUI, streams            |
| <b>03 — Developer Guide</b>         | anyone who wants to build/compile/extend  | toolchain, build, app model, extending the ABI, conventions, debugging |
| <b>04 — User Guide</b>              | anyone who wants to use it                | SD card, desktop, terminal, files, applications, customization         |
| 05 — Circle Changes                 | anyone touching <code>circle/</code>      | the patches of our Circle fork vs upstream <a href="#">Step51</a>      |
| 06 — Jet Browser, 07 — Browser gaps | the browser                               | the Onyx changes to NetSurf, libcss, FreeType; what is missing         |
| EL0 protected mode                  | the execution model                       | how apps moved to EL0, the design                                      |
| Licensing                           | distributors                              | the licences of everything Onyx contains                               |
| Handoff                             | the next session                          | where the work stands, the next tasks                                  |

## 10. Quick genesis (milestones)

1. Taking control of `main()` on top of Circle's `sysinit`; serial console.
2. Replacement scheduler ("shadow" header for `CScheduler`).
3. `VBAR_EL1` vectors, trap frame, system-call path.
4. Per-process page tables (`CAddressSpace`), TTBR0/ASID switching.
5. ELF64 loader → process.
6. Framebuffer (`C2DGraphics`) + `GImage` rendering core (ported from the author's FreeBASIC `SimpleOS`).
7. Compositor + window manager; two animated demos running simultaneously.
8. **Switch to Option C** (EL1 apps + direct call) then **fixed-table ABI**.
9. Onyx desktop (panel + `applist`), stream/stdio subsystem, terminal + `/bin`, file manager.

10. Modal dialogs, themes, app-drawn wallpaper, PID management, keyboard layouts, theme editor (ABI v16).
11. Networking: WLAN bring-up + TCP/IP on the primary core, TCP socket calls (ABI v21), an IRC client, a `/bin/net` tool, and NTP clock sync.
12. The modernised CDE desktop (v64: menu bar, dock, workspaces), the wtk toolkit; the preemptive scheduler; sound on core 1, app cores 2–3 (v51), the GPU (v52); Jet Browser (NetSurf, QuickJS); the office suite, the emulators, Koton; threads (v67); the network on core 3; the `RAM:` volume (v71); the packages.
13. **Every app at ELO** (v73–v74, 2026-10): system calls through the same table, per-process handles, every kapi pointer checked; a fault kills the app, not the machine.